

Intelligent Systems

Solving problems by searching



Definition of a search problem

Search problem = $\langle S, S_0, G, A, T, C \rangle$

- S** State space - a set of possible states $S = \{S_0, S_1, \dots, S_n\}$.
- S_0** Initial state $S_0 \in S$.
- G** A set of goal states $G \subset S$.
Reaching any of the given goals solves the problem.
- A** A set of available actions $A = \{a_0, a_1, \dots, a_m\}$.
Can depend on the current state: $s_i \in S, s_i \mapsto A_i \subset A$
- T** Transition model: $(s_i, a_k) \mapsto S_{i+1}$
- C** Cost function: $(s_i, a_k, s_{i+1}) \mapsto \mathbb{R}$

An example of a search problem

The 8-puzzle game

Can be defined as:

- The state space: $9! = 362880$ states.
- Any initial state (only half are reachable).
- Actions: left, right, up, down.
- Transition model: permutation of the blank square (0) with an adjacent square.
- Goal state: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Cost: each action costs 1.

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

State space transitions

Represented as a graph $\langle V, E \rangle$.

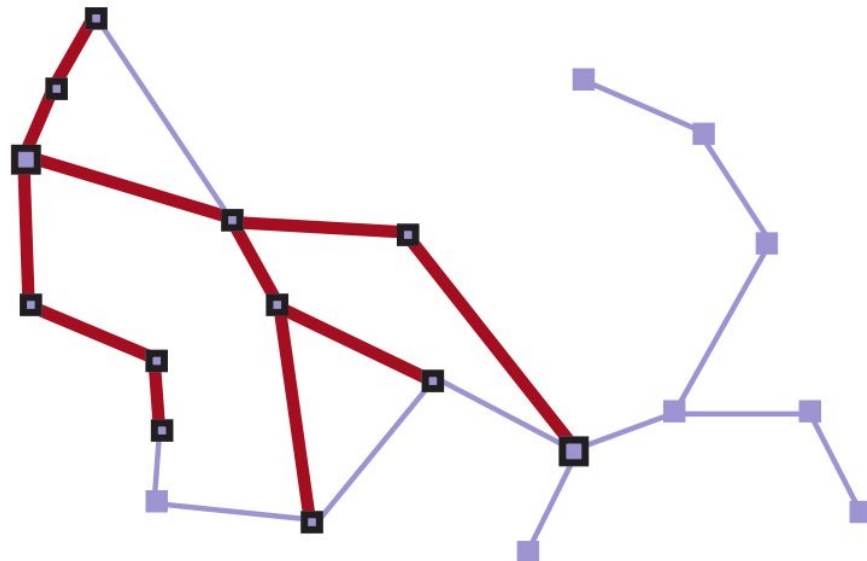
- Vertices V represent states.
- Edges E represent state transitions.

Edges can be directed or undirected.

- Directed - actions are not reversible.
- Undirected - actions are reversible.

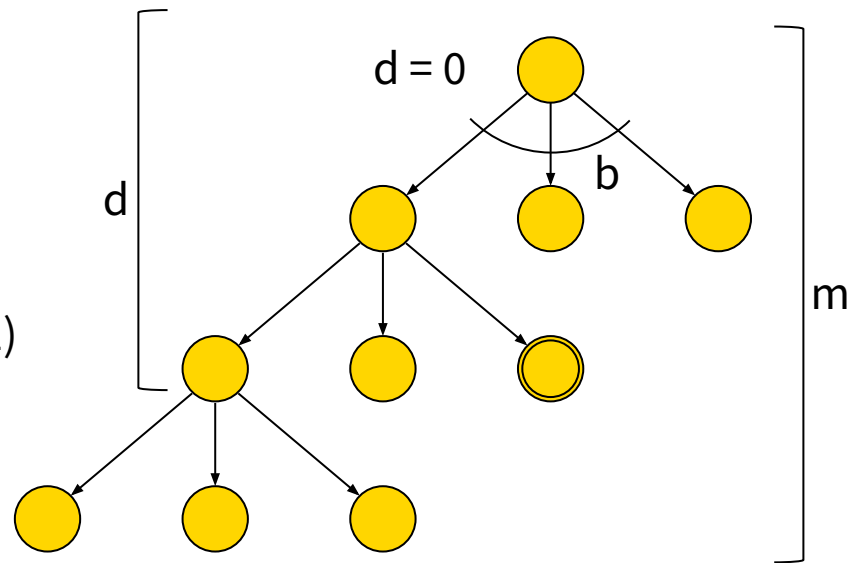
A tree.

- A connected acyclic subgraph with a root node (the initial state).



Search tree

- Branching factor **b** - the number of branches.
- Solution depth **d** - closest solution.
- Maximum depth **m** - if no solution.
- The size of the space: $(b^{m+1} - 1) / (b - 1)$
- The space to be searched: $(b^{d+1} - 1) / (b - 1)$



Search algorithms

Uninformed search

- Depth-first search
- Breadth-first search
- Uniform-cost search (Dijkstra's algorithm)
- Iterative deepening
- Bidirectional search

Informed search

- Greedy best first search
- A*, IDA*

Search algorithm performance

- **Completeness**

Is the algorithm guaranteed to find a solution if there is one and reports a failure if the solution does not exist?

- **Cost optimality**

Does it find a solution with the lowest cost?

- **Time complexity**

How long does it take to find a solution?

- **Space complexity**

How much memory is needed to perform search?

Data structures needed to implement search

```
node {  
    state;           -- the state to which the node belongs;  
    parent;         -- the node in the tree that generated this node;  
    action;         -- the action that was applied to the parent to generate this node;  
    cost;           -- the total cost from the initial state to this node.  
}
```

```
queue {  
    empty()  $\square$  true/false;  -- is the queue empty?  
    pop()  $\square$  node;           -- remove and return the top node;  
    top(node)  $\square$  node;        -- return the top node;  
    add(node);               -- add a node to the top;  
}
```

Three kinds of queues:

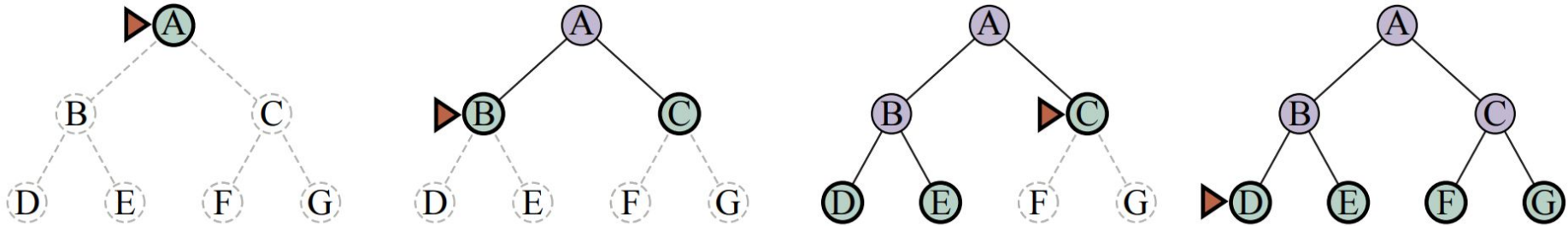
- FIFO (first-in-first-out)
- LIFO (last-in-first-out) / stack
- Priority queue

Uninformed search



Breadth-first search

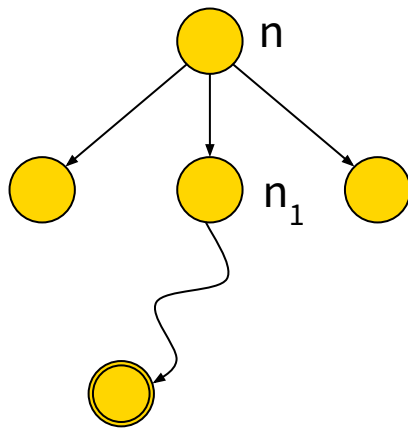
- Guarantees an optimal solution (cost-optimal).
- Searches the whole level before moving to the next one.
- Time complexity: $O(b^d)$
- Space complexity: $O(b^d)$
- Implemented using FIFO (first-in-first-out).



Depth-first search

- Does not guarantee an optimal solution (not cost-optimal).
- Can get stuck in a cycle (if not depth-limited).
- Time complexity: $O(b^m)$
- Space complexity: $O(m)$
- Can easily be implemented using recursion.

```
depth_first(s):  
    if is_goal(s) return [s]  
    for each child n of s:  
        depth_first(n) □ path  
        if path != None return [s] + path  
    return None
```



Limited depth-first search

- We set the depth limit **L** at which the search treats all nodes as leaves.
- This prevents the possibility of endless cycling.
- The solution can be missed if the limit is set too low.
- Useful if we know an approximate length of the solution.

Iterative deepening

- Execute depth-first search in the following way:

```
L = min_depth
do:
    limited_depth_search(s, L) □ solution
    L = L + 1
until solution != None
```

- Guarantees an optimal solution + low space complexity.
- Time complexity: $O(b^d)$
- Space complexity: $O(d)$
- Most of the work is done on the bottom level.

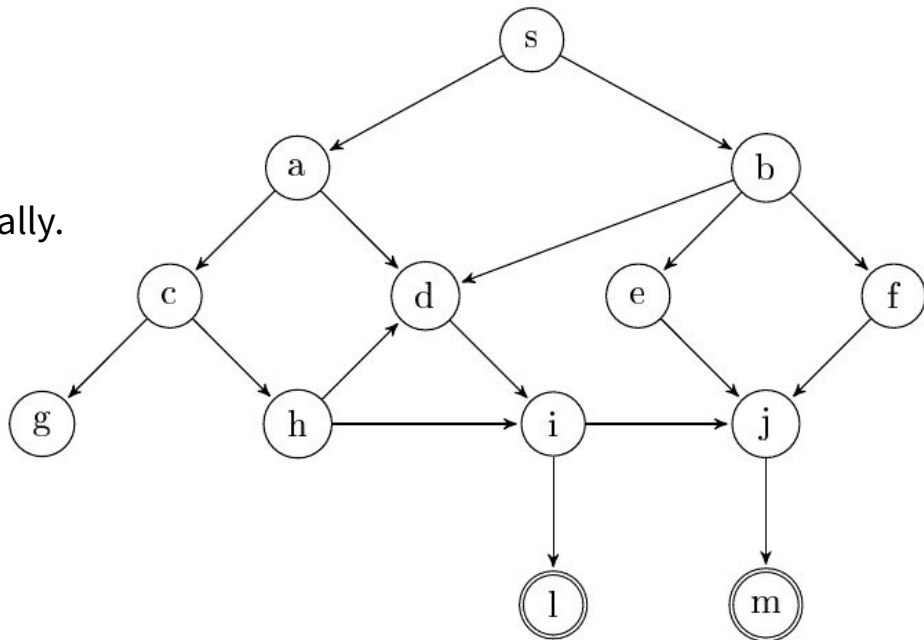
Practical task

State space:

- Let **s** be the initial state.
- Let **m** and **l** be goal states.
- The graph is directed.
- The cost of each transition is 1.
- Assume the nodes get generated alphabetically.

Simulate:

- Depth-first search
- Breadth-first search
- Iterative deepening



Computational tasks

Consider a binary tree ($b = 2$), with solution at $d = 3$.

- How many nodes must in worst case be searched by:
 - Breadth-first search (BFS)? (Answer: 15)
 - Iterative deepening (ID)? (Answer: 26)
- How much more work must ID do in comparison with BFS?
 - Answer: 73%
- Let $b = 11$ and $d = 3$.
 - How much overhead does ID have in comparison with BFS? (Answer: ~10%)

Computational tasks

We used the equation:

$$N_d = (b^{d+1} - 1) / (b - 1),$$

where N_d is the number of nodes of the tree with branching factor **b** and depth **d**.

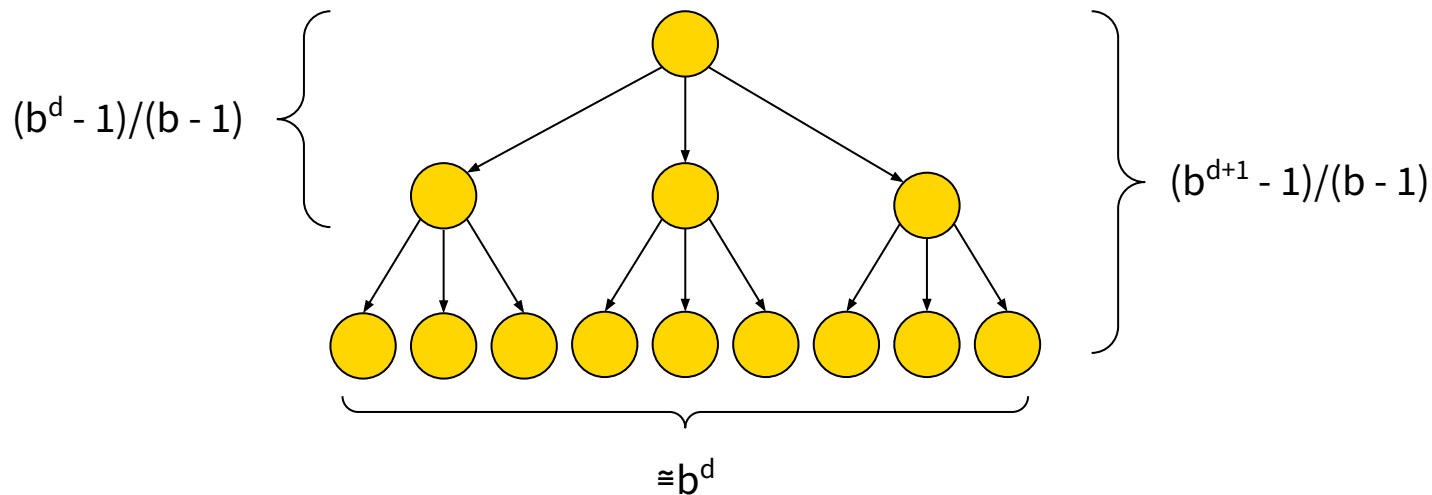
Show that this equation is true.

Computational tasks

- Let the search space be a tree with an unknown branching factor **b**.
- We run breadth-first search algorithm.
- The first solution is found at depth $d_1 = 3$, after 1 second.
- The second solution is found at depth $d_2 = 5$, 15 seconds later.
- What is the branching factor **b**? (Answer: $b = 4$)
- How long will it take to find the third solution at $d_3 = 10$?
(Answer: ~4.5 hours)

Computational tasks

In the previous task, we assumed that for a solution at depth $\mathbf{d}$, BFS has to process $\mathbf{b}^{\mathbf{d}}$ nodes. It was just an approximation, since we don't know where on level $\mathbf{d}$ the solution resides. Show that this approximation is correct.

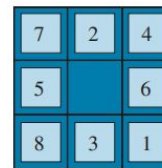
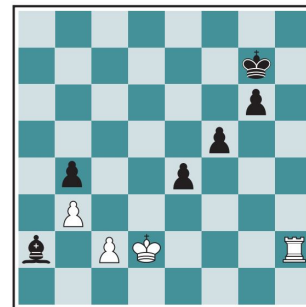
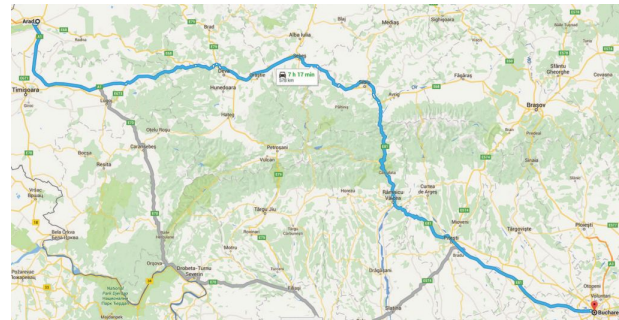


Informed (heuristic) search

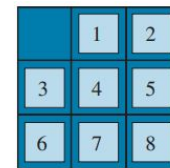


Heuristic function

- Heuristic function h is a domain-specific *hint*.
- $h(n)$ $\square$ estimated cost from node n to a goal state.
- Examples:
 - Euclidean or Manhattan distance in path-finding.
 - (Value of my pieces) - (Value of opponent pieces) in board games.
- What is a possible heuristic function for the 8-puzzle problem?
 - The sum of Manhattan distances of individual tiles to their goal positions.



Start State



Goal State

Greedy best-first search

- Expand the node with the lowest value of: $f(n) = h(n)$.
- Does not evaluate the cost to the expanded node.
- Is complete, but does not guarantee optimal solution.
- Does not guarantee an optimal solution.
- Time and space complexity: $O(b^d)$
- Complexity $O(b^d)$ - the algorithm has to store all the expanded nodes.

A* search

- Expand the node with the lowest value of: $f(n) = g(n) + h(n)$.
- Evaluates the cost of the expanded nodes.
- Value $g(n)$ is the cost from the initial state to **n**.
- If the heuristic function **h** is admissible, optimal solution is guaranteed.
- Time and space complexity: $O(b^d)$
- If $h(n) = 0, \forall n$ then A* behaves as Dijkstra's algorithm.

A* pseudocode

```
function A_Star(start, goal)
    queue ← [(start, 0 + h(start))  # f(start) = g(start) + h(start)
    g[start] ← 0  # The array of best g-values for all visited nodes.
    parent[start] ← None  # Parents of visited nodes.

    while queue not empty:
        node ← pop the node with lowest f-value from the queue
        if node = goal:
            return (path start-node reconstructed from the parent array)

        for each child of node:
            tentative_g ← g[node] + dist(node, child)
            if (child not in g) or tentative_g < g[child]:
                g[child] ← tentative_g
                parent[child] ← node
                queue ← queue + [(child, g[child] + h(child))]

    return failure
```

Admissibility of A*

- An *admissible* heuristic is one that never overestimates the cost to reach the goal.

$$h(n) \leq h^*(n), \quad \forall n$$

- $h^*(n)$ is the actual cost to reach the goal.
- Another name for admissible heuristic function is *optimistic function*.
- If $h(n)$ is admissible, A* guarantees an optimal solution.

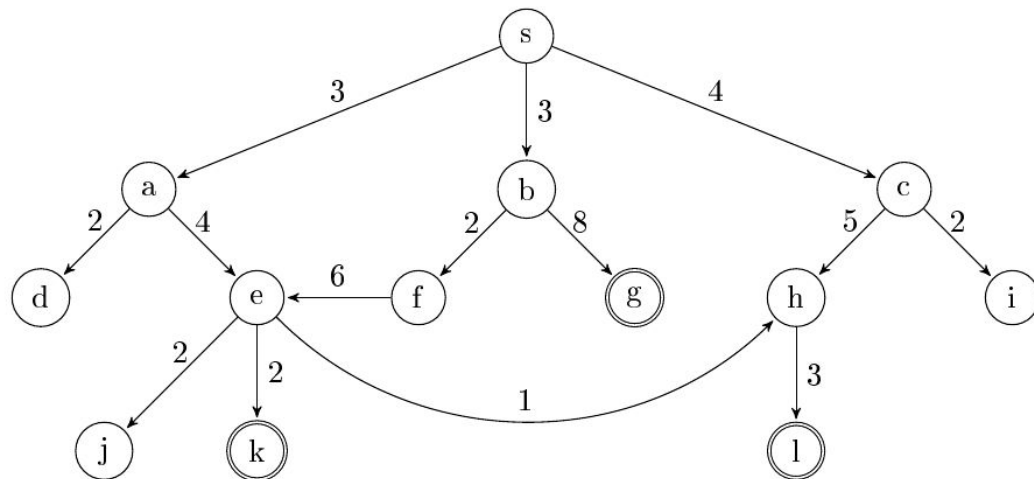
Iterative deepening A* (IDA*)

- Set a limit L for the cost $f(n) = g(n) + h(n)$.
- Expand only the nodes where $f(n) \leq L$.
- If the solution is not found, repeat with a larger L .
- The value of L in the next iteration is the smallest $f(n)$ of the node that was pruned in the current iteration.
- The algorithm doesn't have to store all the reached states in memory.
- Space complexity is $O(d)$ instead of $O(b^d)$.
- Time complexity is $O(b^d)$.

Practical tasks

State space:

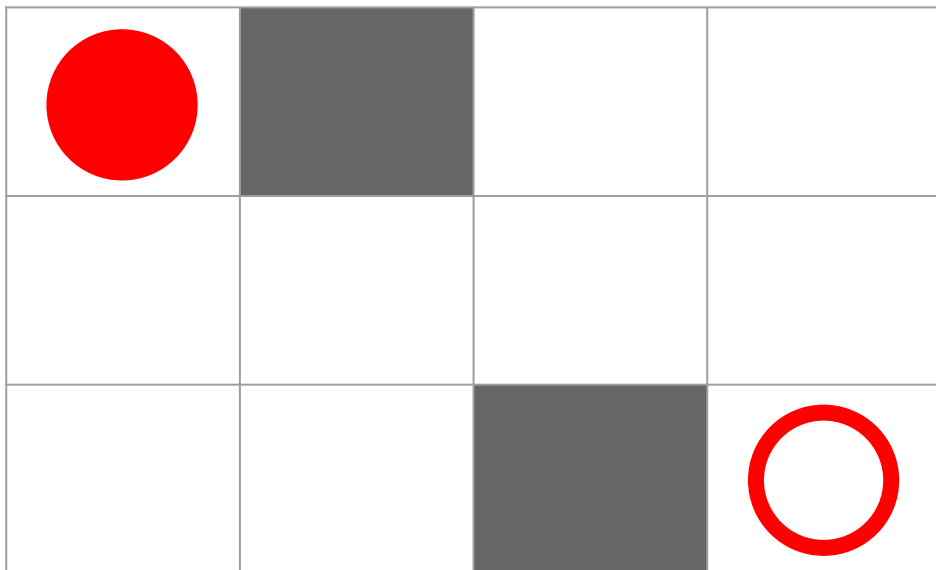
- Let **s** be the initial state.
- Let **k, l, g** be goal states.
- The graph is directed.



- 1) Simulate A* search.
- 2) Correct $h(n)$ with minimal changes to make it optimistic.
- 3) Repeat A* on the optimistic $h(n)$.

vozlišče	s	a	b	c	d	e	f	g	h	i	j	k	l
h	6	5	9	7	10	5	10	0	1	12	12	0	0

Practical tasks



Navigate through the maze using A*.

Use Manhattan distance to the goal as the heuristic function.



Starting position



Goal position

$$h((x, y)) = |x - x_g| + |y - y_g|$$

Adversarial search

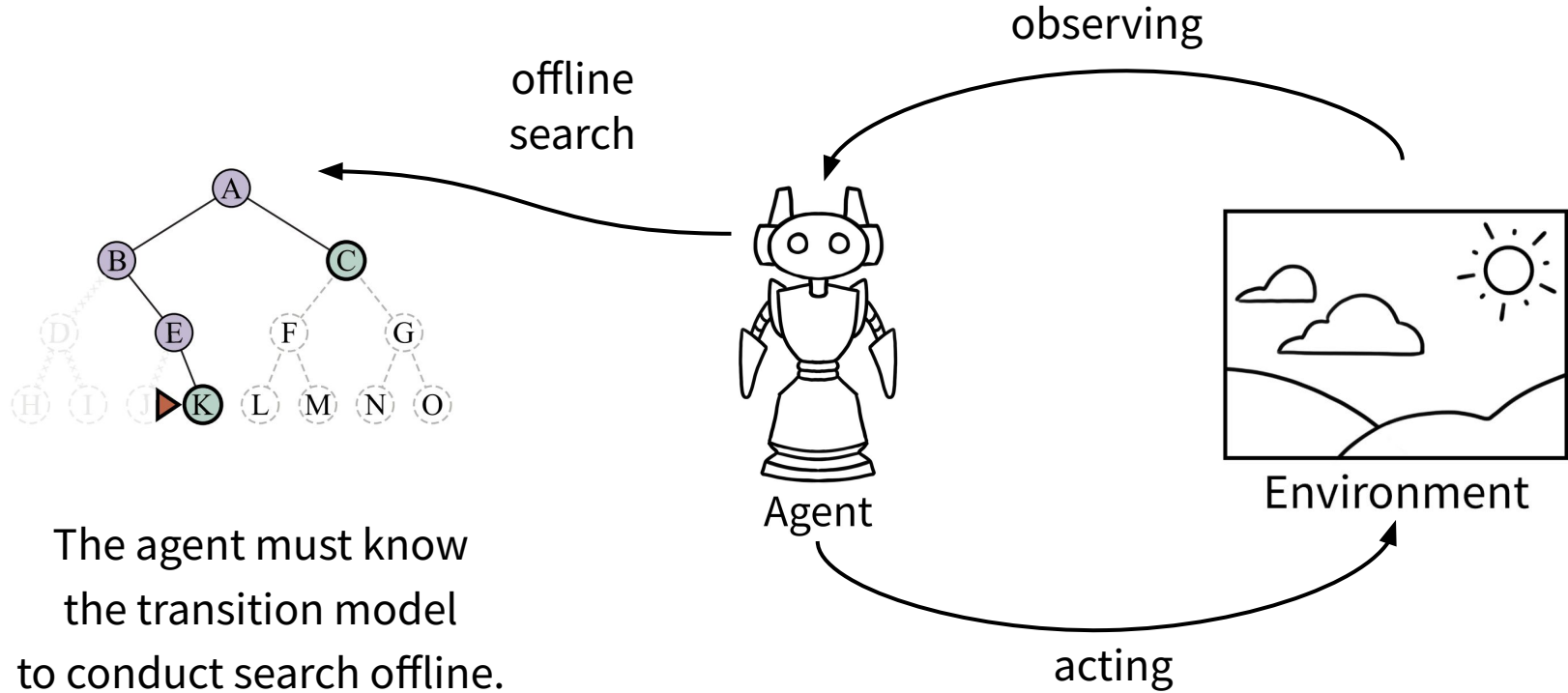


Game theory

The most common type of games approached with classical search:

- Two-player games (usually competitive).
- Turn-based (real-time games → reinforcement learning + neural networks).
- Fully observable (nothing is hidden from either player).
- Deterministic (no dice or random events).
- Zero-sum (one player's gain is another's loss).

Acting vs. “thinking”



Optimal game strategy

- Single-agent domain: the cheapest path from the initial to a goal state.
- Multi-agent domain: A strategy that:
 - minimizes the maximum possible loss,
 - maximizes the minimum guaranteed gain.
 - Under the assumption that the opponent plays optimally.
- If the opponent is not playing optimally, either:
 - miss the opportunities to exploit the opponent's mistakes (possible higher gain),
 - use adaptive strategies (probabilistic reasoning, user modelling, ...)

Maximin and minimax strategies

Maximin (used in decision theory)

- Maximize my profit, assuming the worst will happen.
- Single agent vs. the environment.
- No notion of alternating moves.

$$\overline{v_i} = \max_{a_i} \min_{a_{-i}} v_i(a_i, a_{-i})$$

Minimax (used in game playing)

- Maximize my profit, assuming the other one will try to minimize it.
- Two-players taking turns.
- Adversarial search.

$$\overline{v_i} = \min_{a_{-i}} \max_{a_i} v_i(a_i, a_{-i})$$

Minimax algorithm

- A complete recursive depth-first search of the state space.
- The players are alternating, each player maximizing its own gain.
- From the standpoint of AI:
 - AI is maximizing its score.
 - The opponent is minimizing the AI's score ("his gain is my loss").
- The algorithm computes the minimax values of the successor states.
- The AI chooses the successor with the highest minimax value.
- Time complexity: $O(b^m)$, space complexity $O(m)$.

Minimax pseudocode

```
function MINIMAX(state, depth, maximizingPlayer):  
    if depth == 0 or isTerminal(state):  
        return evaluate(state)  
  
    if maximizingPlayer:  
        maxEval  $\leftarrow -\infty$   
        for each action in actions(state):  
            eval  $\leftarrow$  MINIMAX(result(state, action), depth - 1, False)  
            maxEval  $\leftarrow$  max(maxEval, eval)  
        return maxEval  
  
    else:  
        minEval  $\leftarrow +\infty$   
        for each action in actions(state):  
            eval  $\leftarrow$  MINIMAX(result(state, action), depth - 1, True)  
            minEval  $\leftarrow$  min(minEval, eval)  
        return minEval
```

Alpha-beta pruning

- Minimax search is exponential with the depth of the tree.
- Prune the branches that cannot lead to a better outcome.
- α - the best value for AI, β - the best value for the opponent.
- The search starts with $\alpha = -\infty$, $\beta = +\infty$.
- Maximizer sets the maximum α , the minimizer sets the minimum β .
- A branch where $\alpha \geq \beta$, is pruned.
- “Wisely” ordering the successors can further reduce the size of the search tree.

Alpha-beta pruning pseudocode

```
function ALPHA_BETA(node, depth,  $\alpha$ ,  $\beta$ , maximizingPlayer):  
    if depth == 0 or node is a terminal node:  
        return EVALUATE(node)  
  
    if maximizingPlayer:  
        value  $\leftarrow -\infty$   
        for each child in node.children:  
            value  $\leftarrow \max(\text{value}, \text{ALPHA\_BETA}(\text{child}, \text{depth} - 1, \alpha, \beta, \text{FALSE}))$   
             $\alpha \leftarrow \max(\alpha, \text{value})$   
            if  $\alpha \geq \beta$  break  
        return value  
  
    else:  
        value  $\leftarrow +\infty$   
        for each child in node.children:  
            value  $\leftarrow \min(\text{value}, \text{ALPHA\_BETA}(\text{child}, \text{depth} - 1, \alpha, \beta, \text{TRUE}))$   
             $\beta \leftarrow \min(\beta, \text{value})$   
            if  $\alpha \geq \beta$  break  
        return value
```

Practical task

- Simulate the minimax algorithm on the given tree.
- Which action is chosen by the maximizer in the root node: B or C?
- Simulate alpha-beta pruning and mark the branches that get pruned.

